Concept-to-Animation: Reliable LLM-Generated Manim Explainer Videos

CSE 434 Midterm Check-in Report

Jonah Femiani Koray Bayram

November 13, 2026

Team: Concept-to-Animation **Roles:** *Jonah* — baseline pipeline (LLM client, planner, code generation, render harness, logging, ablation runner), repair and layout loops, analysis, demo. *Koray* — few-shot curation, RAG corpus and retriever, narration. *Both* — human evaluation and reports.

I. Project Overview

Given a short description of a math or CS concept (e.g. "binary search"), our system writes a short explainer video in the style of 3Blue1Brown by having an LLM plan a storyboard and then write Manim Community Edition (ManimCE) code for each scene. LLM-written animation code fails in two characteristic ways: it calls *hallucinated or outdated Manim APIs* (the code crashes), and it produces *bad layouts* (objects overlap or leave the frame). Our research question is: **which generative techniques make LLM-written animation code render reliably and look clear?** We answer it with a cumulative ablation that starts from a zero-shot baseline and adds one technique at a time: error-driven self-repair, few-shot examples, retrieval-augmented generation (RAG) over the ManimCE docs, and a layout-repair loop driven by a bounding-box checker. This connects directly to course topics on prompting, in-context learning, RAG, iterative self-refinement, and LLM code generation. The deliverable is a measured results table, not just a demo, so every LLM call and render attempt is logged.

II. Dataset

Concepts. We curated 6 concepts in three difficulty tiers of two each, chosen by how directly the idea maps to pictures: *concrete* (Pythagorean theorem, derivative as slope), *structural* (matrix multiplication, binary search), and *abstract* (recursion, Big-O notation). Each entry is a JSON record with an id, a name, a tier and a one-sentence prompt. The tiers let us ask whether performance drops as concepts get less visual.

Storyboards. For each concept the planner produces a storyboard of 3–4 scenes (20 scenes in total), each with a title, a visual description and narration. Storyboards are validated against a schema and cached, so every condition generates code for exactly the same scenes and only the code-generation step differs. All 6 were produced in one batch on 2026-11-10 by the same model.

Retrieval corpus (planned). The ManimCE documentation and example gallery (MIT license), pinned to the same version we render with (v0.22.1), chunked and embedded for RAG.

Ethics. The data contains no personal information. The main risk is that a fluent, polished video can teach *wrong* math; render success does not catch this, so we add human ratings of mathematical correctness. Generated videos will be labeled as AI-generated.

III. Methods

Figure 1 shows the current system (v1). A **planner** turns the concept into a storyboard. A **code generator** writes one ManimCE `Scene` subclass per storyboard scene; its prompt states the pinned Manim version, forbids the older ManimGL library, and gives the frame bounds ($x \in$

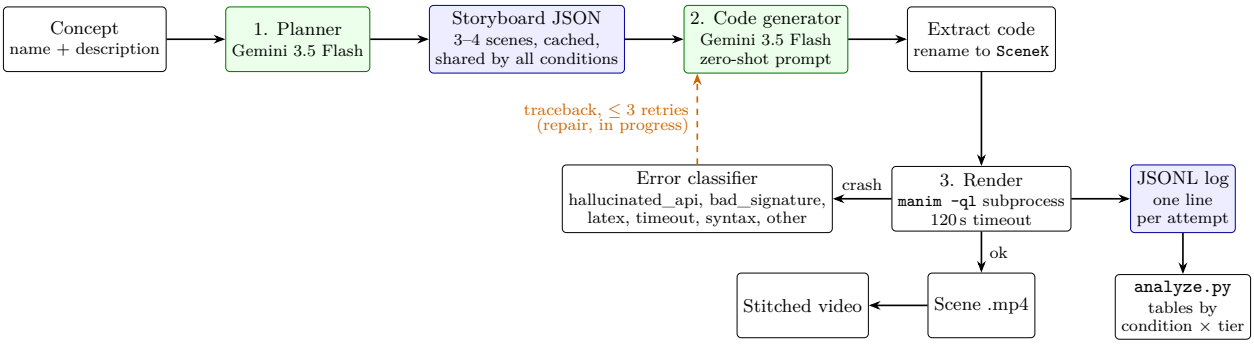


Figure 1: System design v1. Green boxes are LLM calls, blue boxes are stored artifacts. The solid path is the implemented zero-shot baseline; the dashed orange edge is the error-repair loop (`zero_shot_repair`), which feeds the classified traceback back to the code generator. Few-shot examples, RAG context and a layout-repair loop will attach to the code generator in later versions.

$[-7.1, 7.1]$, $y \in [-4, 4]$). Post-processing is deliberately minimal (extract the code block, rename the class) so it cannot silently fix the errors we are measuring. Each scene is **rendered** in a separate subprocess with a 120 s timeout; we check that the expected video file was actually written, because Manim exits successfully even when the named scene does not exist. Failures are **classified** from the traceback into `hallucinated_api`, `bad_signature`, `latex`, `timeout`, `syntax` or `other`. One subtlety: Manim's `Mobject` invents any `get_*` method on demand, so a hallucinated call like `axes.get_secant_line(...)` fails with a `TypeError` rather than an `AttributeError`; our classifier recognizes this pattern as a hallucinated API. Every attempt is written as one JSONL line (tokens, latency, error, code path), and responses are cached by prompt hash so re-analysis costs nothing.

Experimental design. The full study is a cumulative ablation over six conditions: `zero_shot`, `zero_shot_repair`, `few_shot`, `rag`, `rag_repair` and `rag_repair_layout`. Temperature is fixed at 0.2, conditions without repair get exactly one render attempt, and repair conditions get up to three extra attempts. Metrics are computed per scene and aggregated by condition and tier: render success rate, first-try success rate, average repair iterations, layout-defect rate, the error-type breakdown, and human 1–5 ratings of correctness and clarity.

Changes since the proposal (approved by Dr. Zhang). (1) *Models:* the proposal compared GPT with an open model (LLaMA-2/Qwen). We now use Gemini 3.5 Flash for both planning and code generation. We tried Qwen2.5-Coder (7B and 3B, 4-bit via Ollama) and its Manim code was too poor to make the comparison informative; open models remain wired into our LLM interface as optional extras. (2) *No LangChain:* plain Python gives us exact logging of every prompt, reply and token count, which the ablation needs. (3) *6 concepts instead of 40:* each concept costs several LLM calls and renders per condition, and every rendered video needs human rating; 6 balanced concepts (2 per tier, 20 scenes) let us run and hand-rate every condition rather than a few conditions on many concepts. One caveat: Google bills our `gemini-3.5-flash` requests against a `gemini-3.6-flash` quota, so the name may be an alias for a newer model. We log the provider-reported model version on every call to detect drift.

IV. Preliminary Results

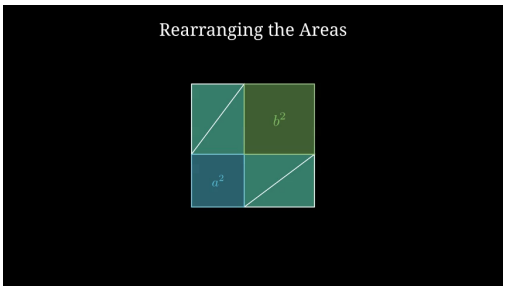
The baseline pipeline runs end to end. Our first zero-shot run stopped at the Gemini free tier's 20-requests-per-day limit after 11 of the 20 scenes in our final concept set (8 of those 11 rendered); we have since moved to a paid key for the full runs. Table 1 reports both conditions on all 20

scenes.

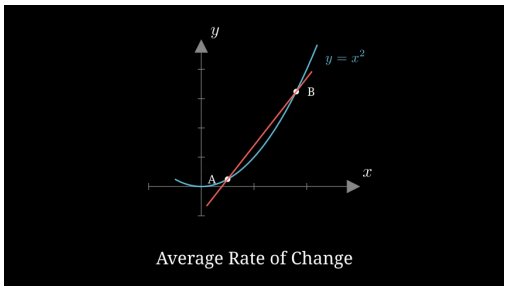
	Automatic (all 20 scenes)			Human, 1–5 (rendered scenes)	
Condition	Render success	First-try success	Avg. repairs	Correctness	Clarity
`zero_shot`	? (?/20)	?	–	?	?
concrete (7)	?	?	–	?	?
structural (6)	?	?	–	?	?
abstract (7)	?	?	–	?	?
`zero_shot_repair`	? (?/20)	?	?	?	?
concrete (7)	?	?	?	?	?
structural (6)	?	?	?	?	?
abstract (7)	?	?	?	?	?

Table 1: Gemini 3.5 Flash on all 20 scenes (6 concepts). First-try success equals render success for `zero_shot`, which gets one attempt; for `zero_shot_repair` it is the same first attempt, so the gap between the two columns is what repair adds. Avg. repairs counts extra attempts over scenes that eventually rendered. Human scores are the mean of both raters over rendered scenes only (a scene that fails to render has no video to rate), rated blind to condition.

Error types. All three failures were API misuse, not syntax or LaTeX problems: two `hallucinated_api` (`axes.get_secant_line(x, curve, dx=...)` and `get_tangent_line(..., graph=...)`, which do not exist in ManimCE) and one `bad_signature` (a keyword argument the real class does not accept, `grid_stroke_width`). This supports our premise: the model writes plausible Python but mixes in API names from older Manim versions or invents them. Since each error message names the bad call, these are exactly the failures a traceback-driven repair loop should fix, and also what RAG over the real docs should prevent.



(a) Pythagorean theorem, scene 3 (rendered).



(b) Derivative as slope, scene 1 (rendered).

Figure 2: Final frames from two zero-shot scenes that rendered on the first try.

Qualitative examples. *Success:* all four Pythagorean-theorem scenes rendered on the first try. Scene 3 (Fig. 2a) rearranges four copies of the triangle inside an $(a + b)$ square to expose the a^2 and b^2 regions, a clean and correct version of the classic proof. *Contrasting failure:* for the derivative, scene 1 (Fig. 2b) correctly draws a secant through two points on $y = x^2$, but scenes 2 and 3, which must *animate* the secant approaching the tangent, both crashed. The model reached for convenience methods (`get_secant_line`, `get_tangent_line`) that sound right but are not in ManimCE. Static drawings succeed; scenes that need an updater-driven animation are where hallucinated APIs appear.

Human ratings. [TODO: correctness and clarity (1–5) from both raters on the rendered midterm videos.]

V. Next Steps

- **Finish the midterm comparison:** complete `zero_shot` on all 20 scenes and run `zero_shot_repair` (traceback in, fixed code out, up to 3 retries), then rate the videos.
- **Few-shot and RAG** (Koray): 3–5 curated modern ManimCE example scenes; a retriever over the pinned ManimCE docs, checked on whether it returns the right classes (e.g. `Axes.plot` for graphs).
- **Layout checker and repair:** instrument `Scene.play` to snapshot every object's bounding box, flag off-frame objects and overlaps (ignoring intentional containment such as a label inside its shape), and describe defects to the LLM in plain language. We will tune the overlap threshold on about ten hand-labeled scenes before trusting the metric.
- **Evaluation:** the full six-condition ablation, results by tier, token cost and latency per condition, human ratings blind to condition with inter-rater agreement, and a success/failure gallery.
- **Narration and demo:** gTTS voiceover via `manim-voiceover`, and a simple web demo (type a concept, get a video).

Open questions for feedback. Is 6 concepts (20 scenes) enough to separate conditions, or should we add more scenes per concept? Is a bounding-box layout check a reasonable proxy for visual clarity, or should clarity rest only on human ratings? Should the stacking order of the ablation be checked with leave-one-out rows?